

OBJECT ORIENTED PROGRAMMING

METHODS, ARRAYS, ARRAYS AS OBJECTS

Method

- ☞ A program module
- ☞ Contains a series of statements
- ☞ Carries out a task

Execute a method - Invoke or call from another method

Calling method (client method) - Makes a method call

Called method - Invoked by a calling method main() method executes automatically

Other methods are called as needed

A method must include:

- ☞ **Method header** - called a declaration
- ☞ **Method body**
 - ★ Between a pair of curly braces
 - ★ Contains the statements that carry out the work
 - ★ Also called implementation

The method header contains:

- ☞ Optional access specifiers
- ☞ A return type
- ☞ An identifier
- ☞ Parentheses - Might contain data to be sent to the method

Place the entire method within the class that will use it - not within any other method

Access Specifiers

- ☞ Can be public, private, protected, or package
- ☞ public access allows use by any other class
- ☞ Also called access modifiers
- ☞ Methods most commonly use public access

Return type

- ☞ Describe the type of data the method sends back to the calling method

- ☞ If no data is returned to the method, the return value is void.

Method name

- ☞ can be any legal identifier
- ☞ Must be one word
- ☞ No embedded spaces
- ☞ Cannot be a Java keyword

Parentheses

- ☞ Every method header contains a set of parentheses that follow the identifier
- ☞ May contain data to be sent to the method
- ☞ Fully qualified identifier - A complete name that includes the class

Adding parameters to methods

- ☞ **Arguments** - Data items you use in a call to a method
- ☞ **Parameters** - Data items received by the method

Implementation hiding

- ☞ Encapsulation of method details within a class
- ☞ The calling method only needs to understand the interface of the called method.

- ☞ **Interface** - The only part of a method that the client sees or with which it interacts

Creating a method that receives a single parameter

- ☞ Optional access specifiers
- ☞ Return type for the method
- ☞ Method name
- ☞ Parameter type
- ☞ Local name for the parameter

Local variable

- ☞ Known only within the boundaries of the method or block where it is declared.
- ☞ Each time the method executes:
 - ★ The variable is redeclared
 - ★ A new memory location large enough to hold the type is set up and named

- 🔗 A method can require more than one parameter
- 🔗 List the arguments within the call to the method - Separate with commas

Call a method

- 🔗 Arguments sent to the method must match the parameters listed in the method declaration by: Number, Type

Creating methods that return values return statement

- 🔗 Causes a value to be sent from the called method back to the calling method
- 🔗 The return type can be any type used in Java
 - ★ Primitive types
 - ★ Class types
 - ★ void - Returns nothing
- 🔗 Method's type - A method's return type

Chaining method calls

- 🔗 Any method might call any number of other methods
- 🔗 Method acts as a black box
 - ★ Do not need to know how it works
 - ★ Just call and use the result

Declaring and using arrays of objects

Create an array of Employee objects:
Employee[] emp = new Employee[7];

Using the enhanced for loop with objects

- 🔗 Use the enhanced for loop to cycle through an array of objects
- 🔗 Eliminates the need to use a limiting value
- 🔗 Eliminates the need for a subscript following each element
- 🔗 for(Employee worker : emp)

Passing arrays to and returning arrays from methods

- 🔗 Pass a single array element to a method
- 🔗 Same as passing a variable

Passed by value

- 🔗 A copy of the value is made and used in the receiving method.
- 🔗 All primitive types are passed by value.

Reference types

- 🔗 The object holds a memory address where the values are stored

- 🔗 The receiving method gets a copy of the array's actual memory address
- 🔗 The receiving method has the ability to alter the original values in the array elements

Returning an array from a method

- 🔗 A method can return an array reference
- 🔗 Include square brackets with the return type in the method header

EXCEPTION HANDLING

Exceptions

- ★ Unexpected or error conditions
- ★ an abnormal condition that must be properly handled to prevent program termination

Objects

- ★ descend from the Throwable class

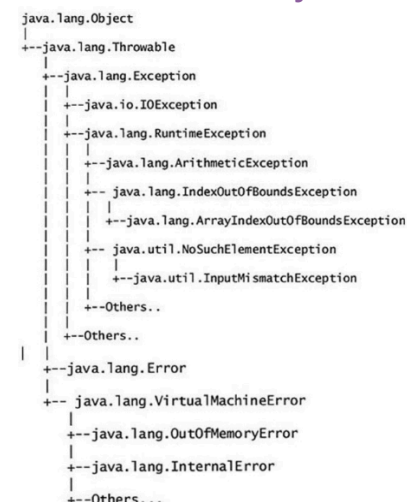
Causes

- ★ The program issues a call to a file that does not exist
- ★ The program attempts to write to a full disk
- ★ The user enters invalid data
- ★ The program attempts to divide a value by 0

Exception handling

- ★ Object-oriented techniques used to manage Exception errors
- ★ Runtime exceptions

The exception and Error class inheritance hierarchy



Error class

- ★ Represents serious errors from which a program usually cannot recover
- ★ Error condition
- ★ A program runs out of memory
- ★ A program cannot locate a required class

Exception class

- ★ Less serious errors
- ★ Unusual conditions
- ★ A program can recover from this type of error

Exception class errors

- ★ An invalid array subscript
- ★ Performing illegal arithmetic operations

Understanding the advantages of Exception Handling

- ★ Before object-oriented programming languages, errors were handled with confusing, error-prone methods
- ★ When any method fails, the program sets appropriate error code
- ★ Difficult to follow
- ★ The application's purpose and intended outcome are lost in a maze of if statements
- ★ Coding mistakes are made because of complicated nesting

Object-Oriented Exception Handling

- ★ Java's object-oriented, error-handling technique
- ★ Statements of the program that do the "real" work are placed together, where their logic is easy to follow
- ★ Unusual, exceptional events are grouped and moved out of the way
- ★ An advantage to object-oriented exception handling is flexibility in handling of error situations
- ★ Appropriately deal with exceptions as you decide how to handle them

try block

- ★ A segment of code in which something might go wrong
- ★ Attempts to execute
- ★ Acknowledges an exception might occur

A try block includes:

- ★ The keyword try
- ★ Opening and closing curly braces

- ★ Executable statements, which might cause an exception

Syntax of try block

```
try {  
    statements  
}
```

catch block

- ★ A segment of code
- ★ Immediately follows a try block
- ★ Handles an exception thrown by the try block preceding it
- ★ Can "catch" an Object of type Exception or an Exception child class

A catch block includes:

- ★ The keyword catch
- ★ Opening and closing parentheses
- ★ An Exception type
- ★ A name for an instance of the Exception type
- ★ Opening and closing curly braces
- ★ Statements to handle the error condition

Syntax of a catch block.

```
catch(ThrowableObjectType paramName)  
{  
    //statements to handle the exception  
}
```

Note: If no exception exists inside the try block, the catch block does not proceed. Inside a catch block, you may want to add code to correct the error.

Creating Object-Oriented Programs That Utilizes Exception Handling

Using a try Block to Make Programs "Foolproof"

- ★ It is useful to circumvent data entry errors
- ★ Handle potential data conversion exceptions caused by careless users
- ★ Using a nextLine() call will account for potential remaining characters in the input buffer

Declaring and Initializing Variables in try...catch Blocks

- ★ A variable declared within a block is local to that block - It goes out of scope when the try or catch block ends
- ★ If the conversion fails, an exception is thrown

Throwing and Catching Multiple Exceptions

★ You can place multiple statements within a try block – Only the first error-generating statement throws an exception

★ When a program contains multiple catch blocks:

- They are examined in sequence until a match is found for the Exception type
The matching catch block executes
Each remaining catch block is bypassed
- “Catch-all” block
- Accepts a more generic Exception argument type: catch(Exception e)
- Unreachable code - Program statements that can never execute under any circumstances
- It is poor style for a method to throw more than three or four Exception types

finally block

★ Use for actions you must perform at the end of a try...catch sequence

★ Use to perform cleanup tasks

★ Executes regardless of whether the preceding try block identifies an exception

When the try code fails:

★ It throws an exception

★ The Exception object is caught

★ The catch block executes - Control passes to statements at the end of the method

Reasons the final set of statements might never execute:

★ An unplanned exception might occur

★ The try or catch block might contain a `System.exit();` statement

The try block might throw an Exception object for which you did not provide a catch block

★ Program execution stops immediately

★ The exception is sent to the operating system for handling

★ The current method is abandoned

When the finally block is used, finally statements execute before the method is abandoned. The finally block executes no matter what outcome of the try block occurs

★ The try ends normally

★ The catch executes

★ The exception causes the method to abandon prematurely

Specifying the exceptions that a Method can Throw

★ If a method throws an exception that it will not catch but will be caught by a different method, use the keyword `throws` followed by the Exception type in the method header

★ **Exception specification** - Lists exceptions that a method may throw

Every Java method has the potential to throw an exception

★ For most Java methods, do not use the throws clause

★ Let Java handle any exception by shutting down the program

★ Most exceptions never have to be explicitly thrown or caught

Unchecked exceptions

★ Inherit from the Error class or the RuntimeException class

★ You are not required to handle these exceptions

★ You can simply let the program terminate

★ An example is dividing by zero

Checked exceptions

★ Programmers should anticipate checked exceptions

★ Programs should be able to recover from them

If you throw a checked exception from a method, you must do one of the following:

★ Catch it

★ Declare the exception in the method header's throws clause

To use a method to its full potential, you must know: Method's name & Method's return type

- ☆ Type and number of arguments the method requires
- ☆ Type and number of exceptions the method throws

Tracing Exceptions through the Call Stack

Call stack - The memory location where the computer stores the list of method locations to which the system must return

When a method throws an exception:

- ☆ The exception is thrown to the next method up the call stack
- ☆ Methods are allowed to handle exceptions wherever the programmer has decided it is most appropriate
- ☆ Including allowing the operating system to handle the error

printStackTrace() method

- ☆ Displays a list of methods in the call stack
- ☆ Determines the location of an exception
- ☆ Is not placed in a finished program - Most useful for diagnosing problems

Creating your Own Exception Classes

- ☆ Java provides over 40 categories of Exceptions
- ☆ Java allows you to create your own Exception classes - Extend a subclass of Throwable

Exception class constructors:

- ★ Exception()
- ★ Exception(String message)
- ★ Exception(String message, Throwable cause)
- ★ Exception(Throwable cause)

INHERITANCE AND POLYMORPHISM

Inheritance

- ☆ A mechanism that enables one class to inherit both the behavior and the attributes of another class
- ☆ Apply your knowledge of a general category to more specific objects

Unified Modeling Language (UML)

- ☆ Consists of many types of diagrams

Class diagram

- ☆ A visual tool
- ☆ Provides an overview of a class

Use inheritance to create a derived class

- ★ Saves time
- ★ Reduces errors
- ★ Reduces the amount of new learning required to use a new class

Inheritance Terminology

Base class (Superclass/Parent class)

- ☆ Used as a basis for inheritance

Derived class (Subclass/Child class)

- ☆ Inherits from a base class
- ☆ Always "is a" case or an example of a more general base class

Keyword extends

- ☆ Used to achieve inheritance in Java
- ☆ *public class EmployeeWithTerritory extends Employee*
- ☆ Inheritance is a one-way proposition - A child inherits from a parent, not the other way around
- ☆ Subclasses are more specific
- ☆ instanceof operator

Overriding Superclass Methods

Create a subclass by extending an existing class

- ☆ A subclass contains data and methods defined in the original superclass
- ☆ Sometimes superclass data fields and methods are not entirely appropriate for subclass objects

Polymorphism - The same method name is used to indicate different implementations

Override the method in the parent class -

Create a method in a child class that has the same name and parameter list as a method in its parent class

Subtype polymorphism - The ability of one method name to work appropriately for different subclass objects of the same parent class

Override annotation (@Override) - Tells compiler you are overriding a parent class method

✿ When you instantiate an object that is a member of a subclass, you call at least two constructors:

- ★ constructor for the base class
- ★ The constructor for the extended class

✿ The superclass constructor must execute first

✿ When the superclass contains a default constructor, the execution of the superclass constructor is transparent

Using Superclass Constructors that Require Arguments

✿ When you write your own constructor - You replace the automatically supplied version

✿ When extending a superclass with constructors that require arguments - The subclass must provide the superclass constructor with the arguments it needs

✿ When a superclass has a default constructor - You can create a subclass with or without its own constructor

✿ When a superclass contains only constructors that require arguments - You must include at least one constructor for each subclass you create. The first statement within each constructor must call one of the superclass constructors

✿ Call the superclass constructor - super(list of arguments);

✿ Keyword super - Always refers to the superclass

Accessing Superclass Methods

Use the overridden superclass method within a subclass - Use the keyword super to access the parent class method

Comparing this and super

✿ Think of the keyword this as the opposite of super within a subclass

✿ When a parent class contains a method that is not overridden - The child can use the method name with super or this, or alone

Employing Information Hiding Within the Student class:

✿ The keyword private precedes each data field

✿ The keyword public precedes each method

Information hiding

✿ The concept of keeping data private

✿ Data can be altered only by methods you choose and only in ways that you can control

When a class serves as a superclass Subclasses inherit all data and methods of the superclass

✿ Except private members of the parent class are not accessible within a child class's methods

Keyword protected

✿ Provides an intermediate level of security between public and private access

✿ Can be used within its own class or in any classes extended from that class

✿ Cannot be used by "outside" classes

Methods you cannot override

✿ static methods

✿ final methods

✿ Methods within final classes

A Subclass Cannot Override static Methods in Its Superclass

✿ A subclass cannot override methods declared static in the superclass

✿ A subclass can hide a static method in the superclass by declaring a static method with the same signature as the static method in the superclass

- ★ Then call the new static method from within the subclass or in another class by using a subclass object
- ★ Within the static method of a subclass, you cannot access the parent method using the super object

✿ Although a child class cannot inherit its parent's static methods, it can access its parent's static methods in the same way any other class can

A Subclass Cannot Override final Methods in Its Superclass

✿ A subclass cannot override methods declared final in the superclass

- ❁ final modifier - Does not allow the method to be overridden
- ❁ Virtual method calls
 - ★ Default in Java
 - ★ The method used is determined when the program runs
 - ★ The object type might not be known until the method executes
- ❁ Advantages to making the method final:
 - ★ The compiler knows only one version of the method exists
 - ★ The compiler knows which method version will be used
 - ★ A program's performance can be optimized by removing calls to final methods
Inlining the code: each method call is replaced with the expanded code of the method's definition
- ❁ When a class is declared final:
 - ★ All of its methods are final regardless of which access modifier precedes the method name
 - ★ It cannot be a parent class

MULTI LEVEL INHERITANCE

Multilevel inheritance - A class inherits properties from a class which again has inherited properties.

Creating a Multilevel Inheritance Hierarchy in Java

Inheritance involves an object acquiring the properties and behavior of another object. So basically, using inheritance can extend the functionality of the class by creating a new class that builds on the previous class by inheriting it.

Multilevel inheritance is when a class inherits a class which inherits another class.

Abstract classes

- ❖ Cannot create any concrete objects
- ❖ Can inherit
- ❖ Usually has one or more empty abstract methods
- ❖ Use keyword `abstract`

- ❖ Provide superclass from which other objects can inherit
- ❖ Example: `public abstract class Animal`

Abstract method does not have:

- ❖ Body
- ❖ Curly braces
- ❖ Method statements

To create abstract method

- ❖ Keyword `abstract`
- ❖ Header including method type, name, and parameters
- ❖ Include semicolon at end of declaration
- ❖ `public abstract void speak();`

Subclass of abstract class

- ❖ Inherits abstract method from parent
- ❖ Must provide implementation for inherited method
- ❖ Or be abstract itself
- ❖ Code subclass method to override empty superclass method

Using Dynamic Method Binding

- ❖ Every subclass object "is a" superclass member
- ❖ Convert subclass objects to superclass objects
- ❖ Can create reference to superclass object
 - ★ Create variable name to hold memory address
 - ★ Store concrete subclass object
 - ★ Example:
`Animal animalRef;`
`animalRef = new Cow();`

Dynamic method binding

- ❖ Also called late method binding
- ❖ Application's ability to select correct subclass method
- ❖ Makes programs flexible
- ❖ When application executes - Correct method attached (or bound) to application based on current, changing (dynamic) context

Using a Superclass as a Method Parameter Type

- ❖ Useful when you want to create a method that has one or more parameters - That might be one of several types

- ✧ Use dynamic method binding
- ✧ public static void talkingAnimal
(Animal animal)
- ✧ Dog dog = new Dog();
- ✧ talkingAnimal(dog);

Creating Arrays of Subclass Objects

- ✧ Create superclass reference
- ✧ Treat subclass objects as superclass objects
 - * Create array of different objects
 - * Share same ancestry
- ✧ Creates array of three Animal references
- ✧ Animal[] ref = new Animal[3];
- ✧ Reserve memory for three Animal object references

Using Inheritance To Achieve Good Software Design

- ✧ Create powerful computer programs more easily - If components used "as is" or slightly modified
- ✧ Make programming large systems more manageable

Advantages of extendable superclasses

- ✧ Save development time - Much code already written
- ✧ Save testing time - Superclass code already tested
- ✧ Programmers understand how superclass works
- ✧ Superclass maintains its integrity - Bytecode not changed

Creating and Using Interfaces

Multiple inheritance

- ✧ Inherit from more than one class
- Prohibited in Java
- ✧ Variables and methods in parent classes might have identical names
 - * Creates conflict

Interface

- ✧ Alternative to multiple inheritance
- ✧ Looks like a class - Except all methods and data items implicitly public, abstract, and final
- ✧ Description of what a class does
- ✧ Declares method headers

Create an interface

- ✧ Example: public interface Worker

- ✧ Implement an interface - Keyword implements

- ✧ Requires subclass to implement own version of each method
 - ✧ Use interface name in class header
 - ✧ Requires class objects to include code
- ```
public class WorkingDog extends Dog
implements Worker
```

### Abstract classes versus interfaces

- ✧ Cannot instantiate concrete objects of either

#### Abstract classes

- ✧ Can contain nonabstract methods
- ✧ Provide data or methods that subclasses can inherit
- ✧ Subclasses maintain ability to override inherited methods
- ✧ Can include methods that contain actual behavior object performs

#### Interface

- ✧ Methods must be abstract
- ✧ Programmer knows what actions to include
- ✧ Every implementing class defines behavior that must occur when method executes
- ✧ Class can implement behavior from more than one parent

### Creating Interfaces to Store Related Constants

- ✧ Interfaces can contain data fields
- ✧ Must be public, static, and final
- ✧ Interfaces contain constants - Provide set of data that many classes can use without having to redeclare values